

+ 26

The Latency Slippage

Understanding the Hidden Costs of Caching
for Low Latency C++

SANCHIT GUPTA



20
26



Sanchit Gupta

Software developer at Graviton Research Capital, a high-frequency trading firm.



A little about Graviton

Graviton Research Capital - a proprietary quantitative trading firm founded in 2014. The firm runs systematic strategies across multiple asset classes and exchanges, built on an in-house low latency trading platform

What I work on

My day-to-day work is optimising C++ for the lowest possible latency — the runtime and the jitter of production trading processes.

LinkedIn [linkedin.com/in/sanchit-gupta-361581189](https://www.linkedin.com/in/sanchit-gupta-361581189)



What this talk is about

The hardware is guessing — which line you will want next, who else wants it, what is worth keeping. Those guesses are **heuristics** : some documented, some measurable. We will learn about some of them, through one application taken apart and see how it affects overall performance



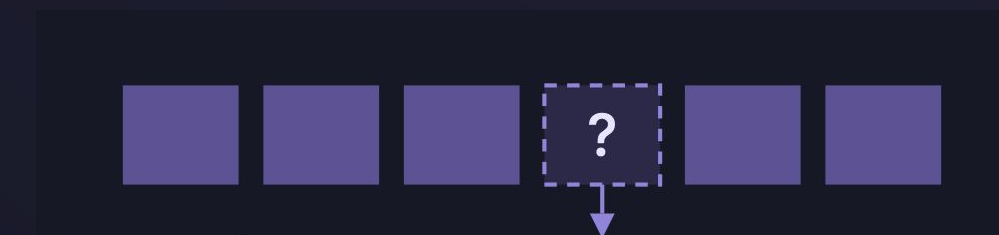
Prediction

Which line will it want next? Guess right and the miss never happens.



Ownership

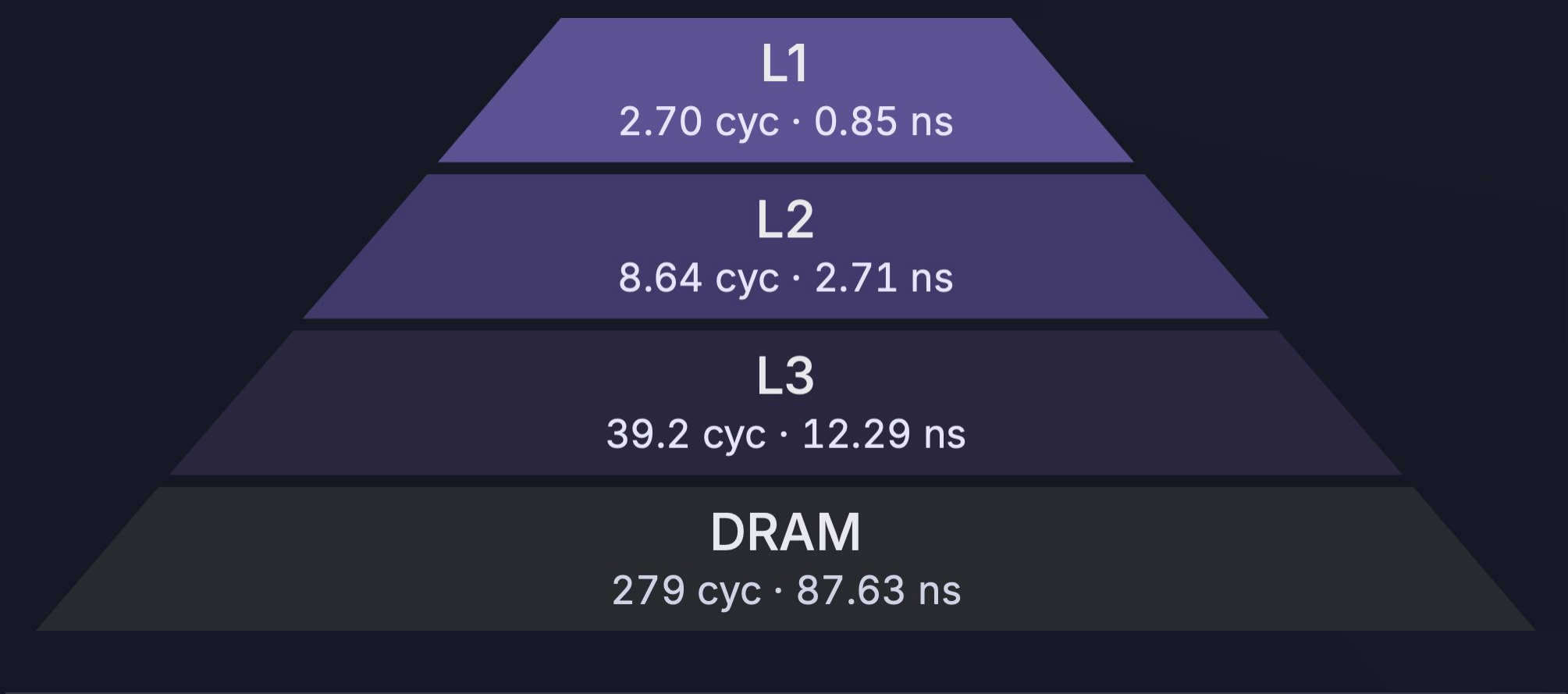
Who holds it, and in what state?
Reading is free. Writing is not.



Retention

The set is full. Which line goes, so this one can come in?

What each level costs



RELATIVE 1 : 3.2 : 14.5 : 103

THE MACHINE

CPU Core i9-14900KS Raptor Cove

Clock 3.1872 GHz TSC core 5.90 GHz

L1d 48 KiB per core

L2 2 MiB per core

L3 36 MiB shared

DRAM 64 GB DDR5 2 × 32 GB · 4,800 MT/s

DIMMs CMK64GX5M2D6000C40 Corsair

4 P-cores online, SMT off, isolated core. Fenced RDTSC;
cyc and ns are both TSC.

What happens when a load is issued in Intel

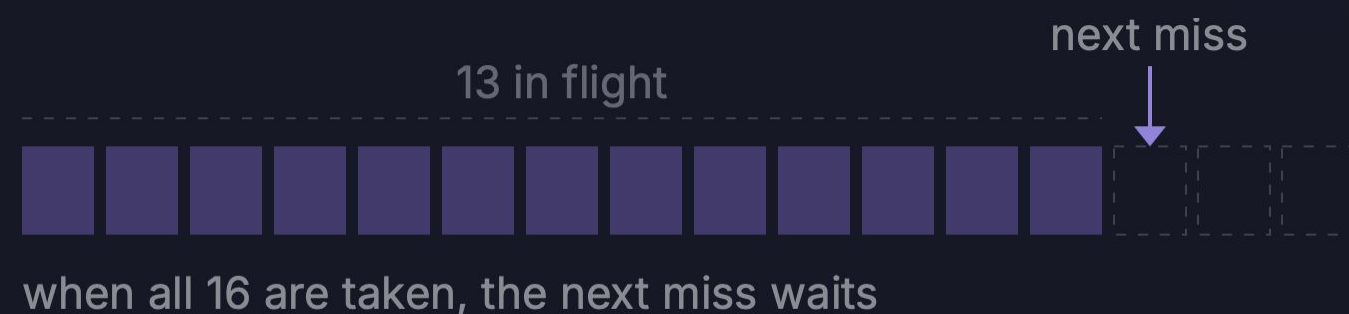


Each hop right is a miss, each hop back is the line arriving. Note that, not every miss would result in a stall.

A miss takes a slot and holds it till the cache line arrives

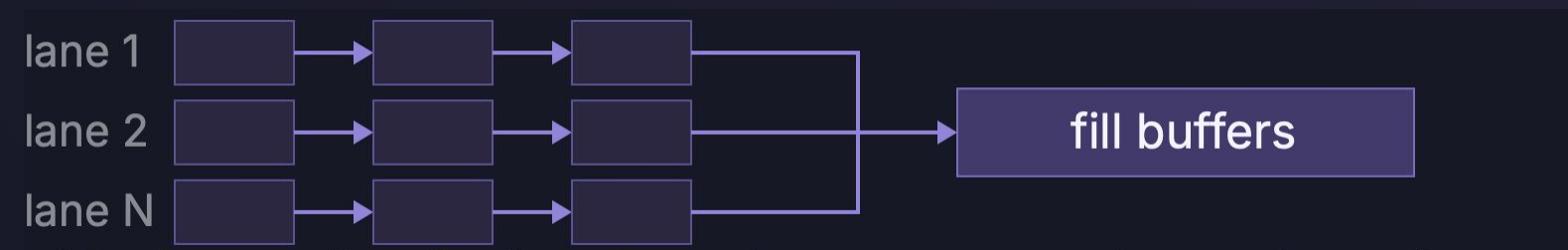
How many misses can one core have in flight?

Sixteen slots



A miss takes a slot and holds it until the line arrives — and a later load to the same line joins that slot rather than taking another.

How you count them



github.com/lemire/testingmlp — every pointer chase hop depends on a memory access and there are N parallel such chases happening.

THE ANSWER depends on no. of FILL BUFFERS

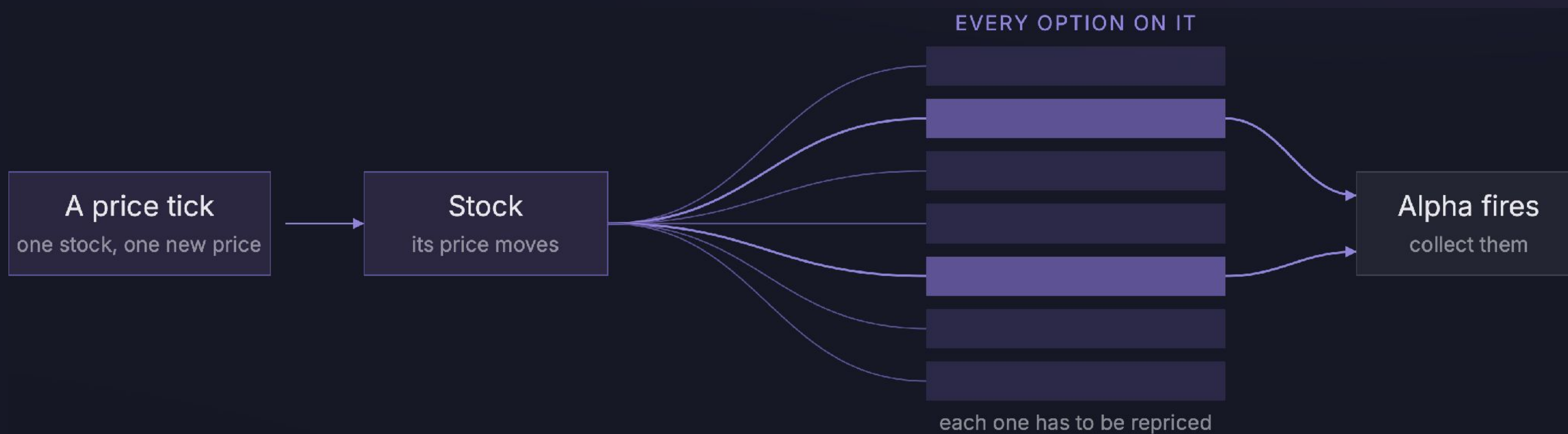
Number of fill buffers is at least 16

speedup over one lane · one lane = 82.1 ns per hop

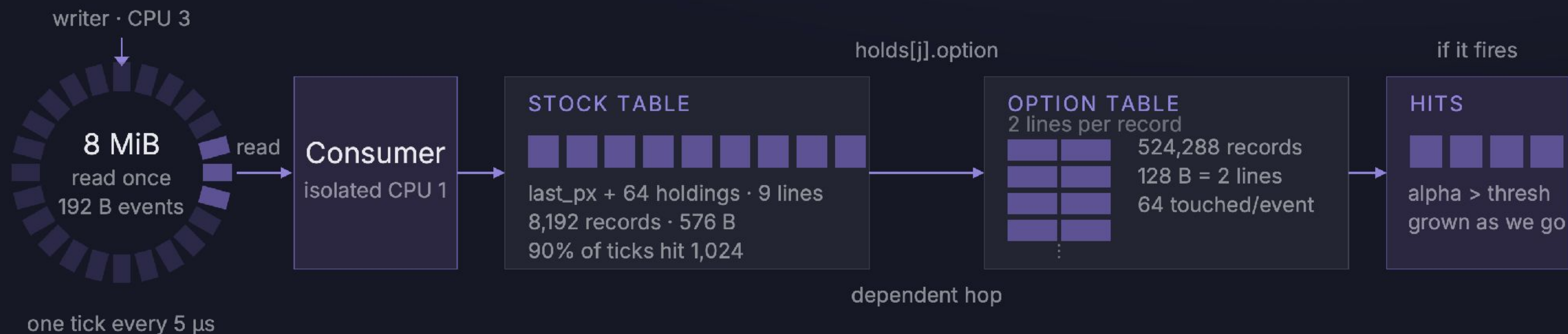


What happened: it tracks the ideal to n lanes, then bends away as the buffers fill — 14.5× **Why that means**
16: timing cannot separate 16 buffers at 90% full from 15 at 97%. So, we count cycles with at least k
misses outstanding : **k=16 is 29.7% of cycles, k=17 exactly zero.**

Simple options pricing application



The ring, the tables, and the walk



One 192 B event every 5 μs, read once from an 8 MiB ring. The stock it names holds 64 options, and every option's address lives *inside* that stock record — so nothing can be fetched until the record has arrived. The options whose alpha clears the threshold are collected as we walk, and that list is the event's result.

Every event touches 140 cache lines

WHERE THOSE LINES GO

The event	3 192 B, off the ring
Its stock record	9 576 B, one lookup
Its 64 options	128 8,192 B, 2 lines each
Total	140 lines 8,960 B per event

Almost all of it is the option walk — 128 of the 140, scattered across half a million records.

WHAT A RUN REACHES

Events measured	200,000
Tables reached	42.7 MiB of 68.5 MiB allocated
Ring on top	8 MiB streaming through
L3 to hold it	36 MiB shared by every core

So the live footprint is about 40% larger than L3. The tables cannot all stay resident, and the ring is pushing them out while it streams.

AGAINST 36 MIB OF L3

50.7 MiB of tables and ring

The consumer

```
while (ev->seq.load(acquire) < base_seq + e) {} // tick

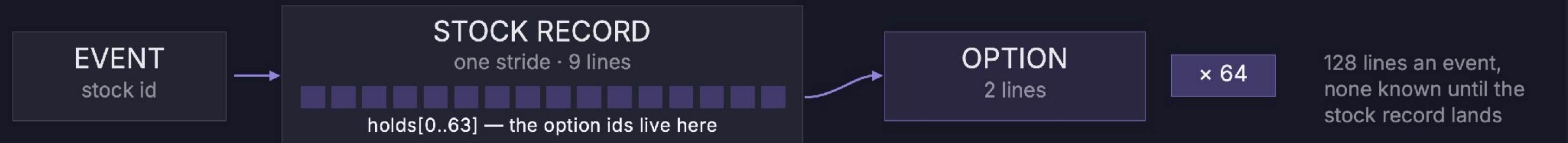
Stock* st = &stock[ev->stock]; // 9 lines
const double move = ev->px - st->last_px;
st->last_px = ev->px;

const size_t na = r.alpha; // 25 FLOPs per option

for (size_t j = 0; j < K; ++j) { // K = 64
    Option* t = &option[st->holds[j].option];
    t->curr_px += move * st->holds[j].delta;
    t->alpha = alpha_of(t->curr_px, t->actual_px,
                       t->aux[0] + t->aux[4], na);
    if (t->alpha > THRESH) list[n++] = st->holds[j].option;
}
```

What to notice in that loop

ONE EVENT, SIXTY-FOUR OPTIONS



The address arrives late

Every option's address lives *inside* the stock record.

But there are 64 of them

The updates are independent, so the out-of-order window can overlap some of them without us paying the penalty of 64 serialised misses.

Memory layout

The data structures are contiguous and number of dependent lookups are already minimised.

The baseline benchmark

nanoseconds



WHERE THE LOADS LAND

L1	222.2 hits per event
L2	19.2
L3	103.7
DRAM	6.6

PER EVENT, MEDIAN 510 ns · 1,012 at the p90 · 8,142 instructions

What can we try next?

Overlap the misses

All the 128 lines don't have to be fetched one at a time. The updates are independent — the only reason they serialise is the order we wrote them in.

one at a time

overlapped



What the loop does

Request one option's lines, run 25 dependent FLOPs, then request the next.

Why that serialises

The alpha chain fills the reorder buffer, so the core never reaches far enough ahead to have many misses in flight.

The fix

Gather a batch of lines first, then compute on the batch.

THE CEILING

no. of fill buffers

Two passes over a batch of D

Before — one interleaved walk

```
for (size_t j = 0; j < K; ++j) {
    Option* t = &option[st->holds[j].option];
    t->curr_px += move * st->holds[j].delta;
    t->alpha = alpha_of(t->curr_px, t->actual_px,
                       t->aux[0] + t->aux[4], na);
    if (t->alpha > THRESH)
        list[n++] = st->holds[j].option;
}
```

After — gather, then compute

```
for (size_t b = 0; b < K; b += D) {
    const size_t hi = std::min(b + D, K);
    for (size_t j = b, k = 0; j < hi; ++j, ++k) {
        Option* t = &option[id[k] =
                       st->holds[j].option];
        nc[k] = t->curr_px
                + move * st->holds[j].delta;
        ac[k] = t->actual_px;
        ax[k] = t->aux[0] + t->aux[4];
    }
    for (size_t k = 0; k < hi - b; ++k) {
        option[id[k]].curr_px = nc[k];
        option[id[k]].alpha =
            alpha_of(nc[k], ac[k], ax[k], na);
    }
}
```

The miss penalty gets parallelised



Nothing in a pass depends on previous pass

The batch's misses go out together, so the limit becomes the fill buffers rather than the arithmetic between them.

$D = 1$ is the loop we started with

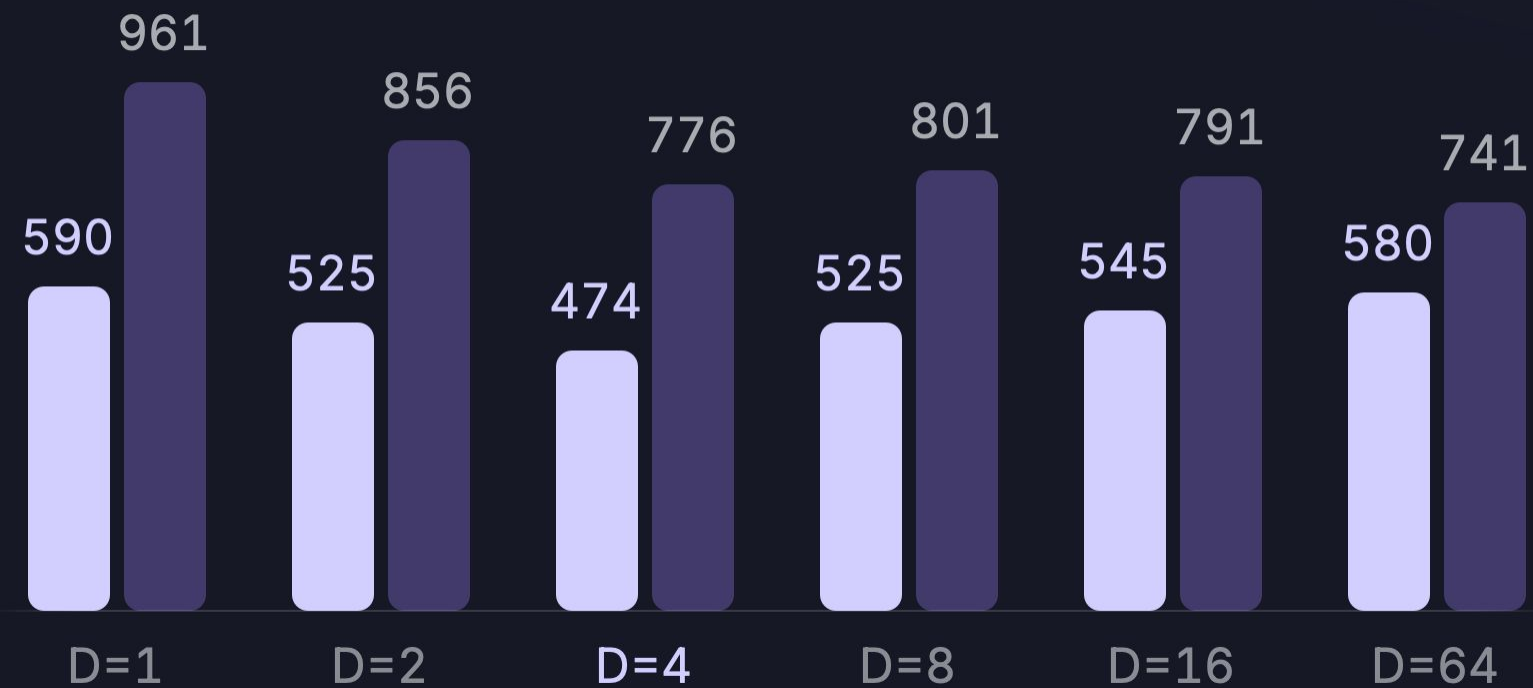
Exactly the interleaved walk from slide 12.

$D = 64$ is fetching everything together

$D = 64$ gathers the whole event before computing any of it.

Performance for different D

nanoseconds per event



D = 4 AGAINST D = 1

p50

590 → 474 ns -19.6%

p90

961 → 776 ns -19.3%

L1 misses

129.6 → 130.0 unchanged

Buffer-full

35 → 36 cycles; 1,001 at D=64

The misses did not go away — the same 130 are simply outstanding at once. D=4 wins the median outright; at p90 only D=64 edges it.

WHY D = 64 LOSES

1,001 cycles stalled on full fill buffers

The number of misses pre event remains same at 130 misses per event, but now they are overlapped. That let the core retire 25% more instructions per cycle, and even with 13% extra instructions, we get a 8% improvement at the median.

CYCLES

−7.9

per cent

1,624 → 1,496 TSC cycles at the median — the overlap is genuine

INSTRUCTIONS

+12.9

per cent

8,142 → 9,191 — the gather arrays, indices and second pass

IPC

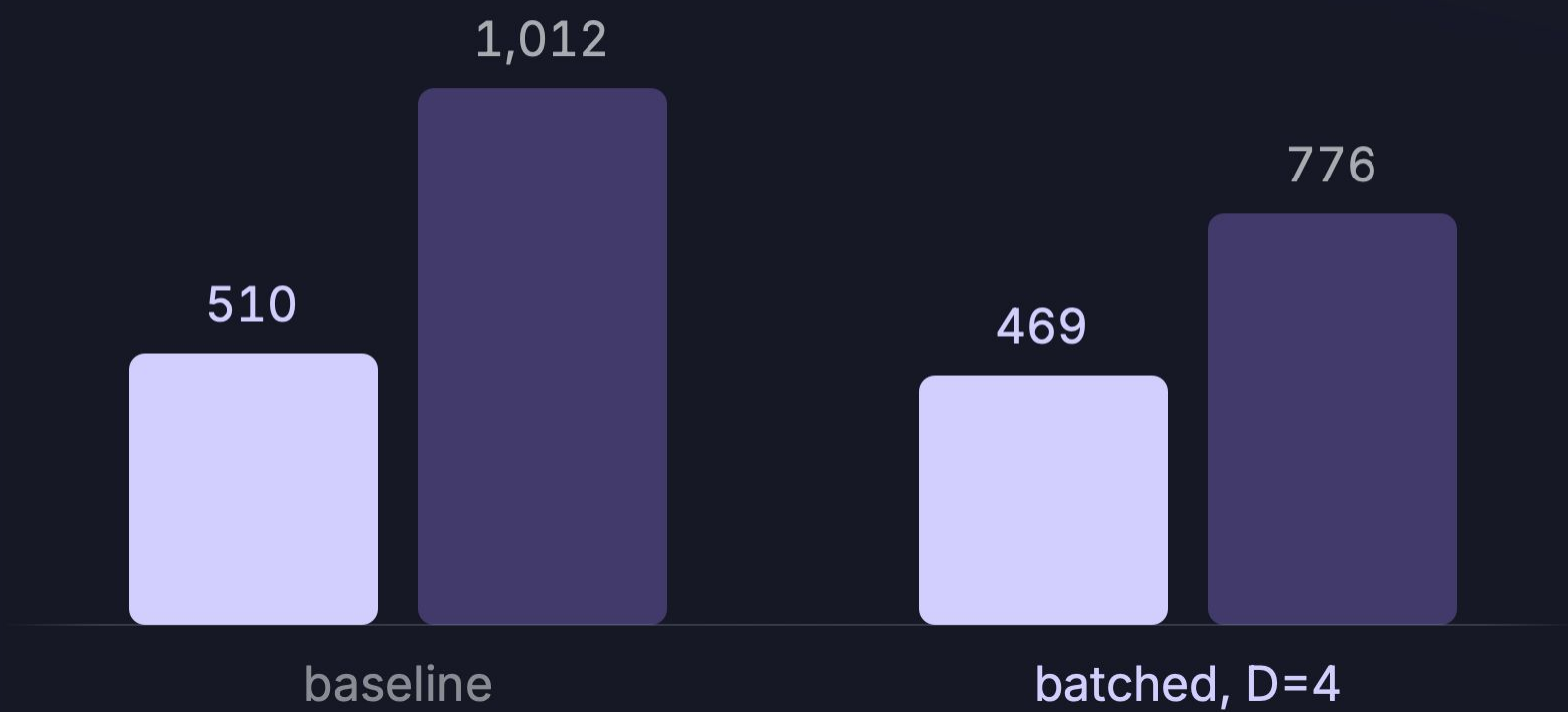
+24.6

per cent

1.98 → 2.47 per core cycle — the same work, denser in time

Conclusion

nanoseconds per event



	p50	p90
BATCHED D=4 AGAINST THE BASELINE		
p50	510 → 469 ns -8.0%	
p90	1,012 → 776 ns -23.3%	
L3 hits	103.7 → 104.0 per event, unchanged	
Instructions	8,142 → 9,191 +1,049 to reorder	

The L3 hits barely move: batching removes no misses, it only stops waiting one at a time. A batch of 2 is too narrow even for that — 525 ns.

What can we try next?

Request the line before use

Start a memory fetch early enough that the line has arrived by the time the load runs.

A prefetch is not a load

It has no destination register, so nothing waits on it and nothing stalls when it misses.

We already know the address

Holding j+D's option id is sitting in the stock record we have read.

THE INSTRUCTION `_mm_prefetch(addr, hint)`

How a prefetch brings the line in



A prefetch that misses L1 takes a fill buffer and holds it until the line arrives — the same 16 per core a demand miss competes for. The hint affects the cache level where the line is brought in

Prefetching the next batch

Before — the batched walk

```
for (size_t b = 0; b < K; b += D) {
    const size_t hi = (b + D < K) ? b + D : K;
    for (size_t j = b, k = 0; j < hi; ++j, ++k) {
        id[k] = st->holds[j].option;
        Option* t = &option[id[k]];
        nc[k] = t->curr_px
            + move * st->holds[j].delta;
        ac[k] = t->actual_px;
        ax[k] = t->aux[0] + t->aux[4];
    }
    // pass two — unchanged
    for (size_t k = 0; k < hi - b; ++k) { ... }
}
```

After — the next batch asked for

```
for (size_t b = 0; b < K; b += D) {
    const size_t hi = (b + D < K) ? b + D : K;
    for (size_t j = b, k = 0; j < hi; ++j, ++k) {
        id[k] = st->holds[j].option;
        Option* t = &option[id[k]];
        nc[k] = t->curr_px
            + move * st->holds[j].delta;
        ac[k] = t->actual_px;
        ax[k] = t->aux[0] + t->aux[4];
    }
    for (size_t j = hi; j < hi + D && j < K; ++j) {
        const Option* t = &option[st->holds[j].option];
        _mm_prefetch((const char*)t, _MM_HINT_T0);
        _mm_prefetch((const char*)t + 64, _MM_HINT_T0);
    }
    // pass two — unchanged
    for (size_t k = 0; k < hi - b; ++k) { ... }
}
```

Asking before the address is needed



Both lines, because the alpha reads both

An option is 128 B: the price on one line, the alpha's state on the other. Both must arrive.

No priming loop needed

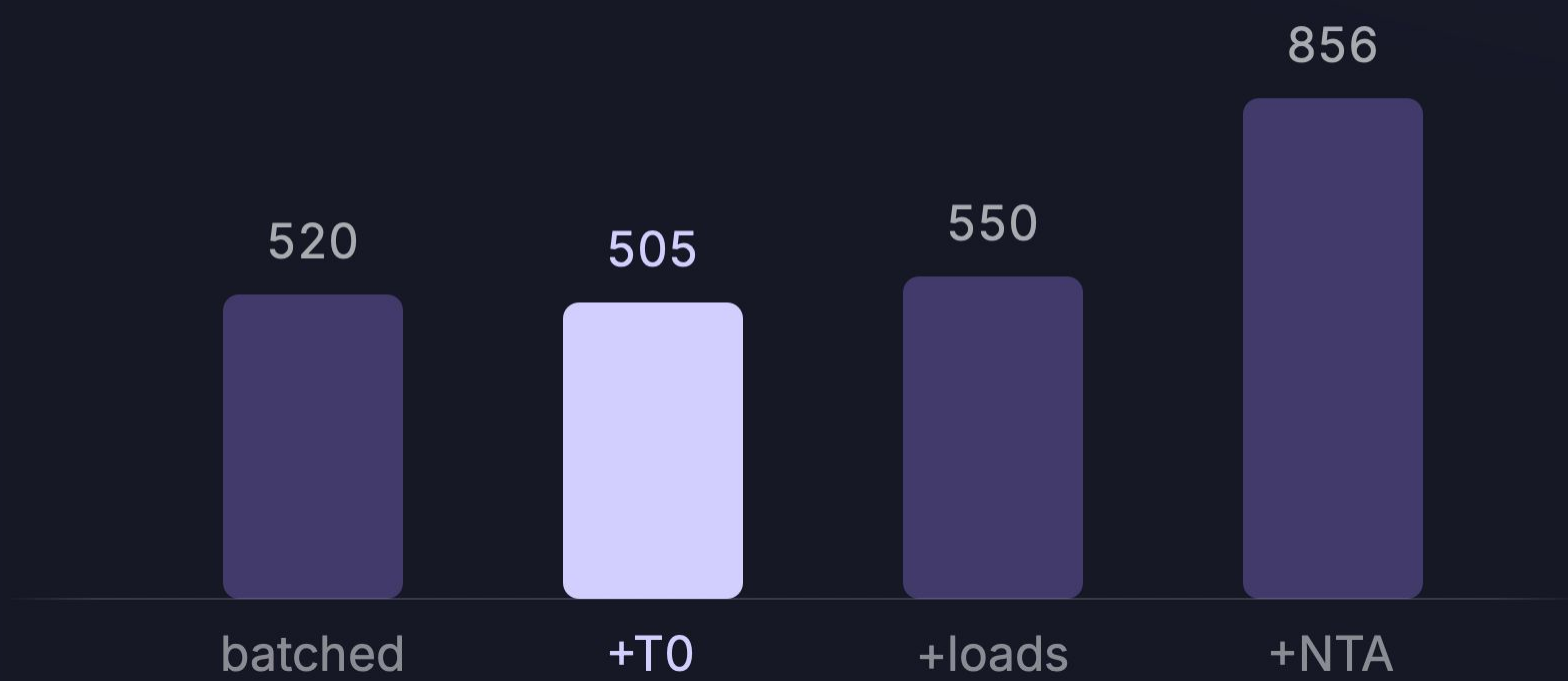
The first batch's own gather is its parallel fetch, so nothing has to be primed. The prefetch starts paying from the second batch on.

The next batch has time to arrive

The alpha compute of the current batch is long enough for the requests issued D ahead of it to complete.

How you ask for the next batch matters

p50, nanoseconds



ON TOP OF BATCHED, D=16

batched	520 ns	p90 776 ns
+T0	505 ns · -2.9%	p90 655 ns · -15.6%
+loads	550 ns · +5.8%	p90 831 ns · +7.1%
+NTA	856 ns · +65%	p90 1,012 ns · +30%

All three ask at the same distance on top of the same batched walk, so the only variable is the instruction. Real loads bring the line and cost more than the prefetch; NTA brings it and then throws it away.

CYCLES STALLED ON AN L1 MISS

1,074 → 292 · and 1,995 with NTA

Benchmarks for varying d

nanoseconds per event

— p50 — p90



Each rung is the one before it plus one thing: baseline, then batched, then batched with the next batch prefetched. Batching buys 15%, the prefetch another 12%. **Too close is as bad as too far** — at D=2 the batch is too narrow; at D=64 there is no next batch. D=4 is read from the ladder run.

So prefetching helps... But some questions are open

How far ahead?

D=4 gives the best median at 394 ns; D=16 the best p90 at 645. Both ends lose — D=2 and D=64 are the worst prefetched rungs. At D=2 the ask is too late: 172 loads an event land on a prefetch still in flight, against 17 at D=4. At D=64 it is too early to help at all.

T0, T1, T2 or NTA?

T0 improves numbers for our use case, but NTA worsens it off NTA hints the cache to evict these lines soon, but our application might reuse these lines in the next event

Any sideeffects ?

Prefetch would bring the requested cache line into our desired cache level. but is this all it does? can it bring some other lines? hw vs. sw prefetching?

Prediction is handled. Eviction is not.

What can we try next?

clflush

How are the lines for eviction chosen? Every line we touch evicts one we may still need, and read-once data evicts information we will want again.

The read once lines

Three lines of every event: a header and two of book. Read once, then dead — but they still hold a way in L1, L2 and L3, and they got it by evicting something.

clflush

Invalidate the line at every level, writing it back first if dirty. It is strongly ordered — fenced against the loads and stores around it — so a run of them serialises.

clflushopt

The same invalidation, weakly ordered. Several can be executed parallelly through the fill buffers.

THE INSTRUCTION `_mm_clflushopt(addr)`

Clflush the event at the end

Before — batched + prefetch, D=4

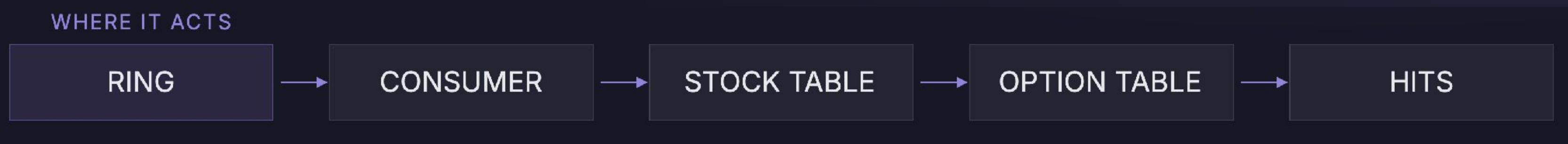
```
for (size_t b = 0; b < K; b += D) {
    const size_t hi = (b + D < K) ? b + D : K;
    // pass one — gather this batch
    for (size_t j = b, k = 0; j < hi; ++j, ++k) { ... }
    // ask for the next batch — prefetch T0
    for (size_t j = hi; j < hi + D && j < K; ++j) { ... }
    // pass two — price and threshold
    for (size_t k = 0; k < hi - b; ++k) { ... }
}
```

After — the spent lines given back

```
for (size_t b = 0; b < K; b += D) {
    const size_t hi = (b + D < K) ? b + D : K;
    // pass one — gather this batch
    for (size_t j = b, k = 0; j < hi; ++j, ++k) { ... }
    // ask for the next batch — prefetch T0
    for (size_t j = hi; j < hi + D && j < K; ++j) { ... }
    // pass two — price and threshold
    for (size_t k = 0; k < hi - b; ++k) { ... }
}

// the event is spent — nobody reads it again
_mm_clflushopt((char*)ev);
_mm_clflushopt((char*)ev + 64);
_mm_clflushopt((char*)ev + 128);
```

Flush out only what is read once



Only what is truly read-once

The ring is read only once and flushing it out would not impact any other path.

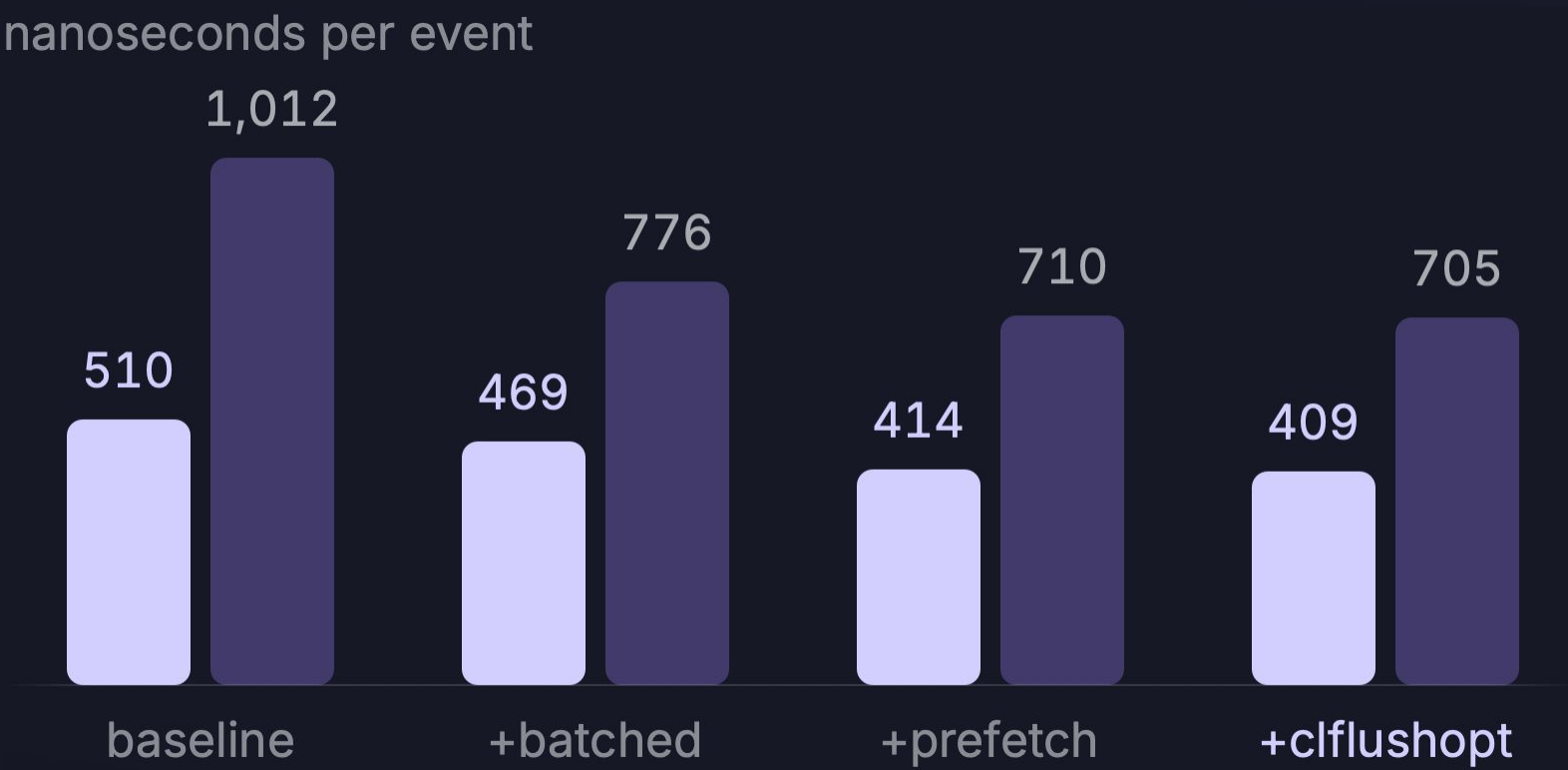
opt, not plain

Weakly ordered, so the three flushes pipeline instead of fencing the loop three times over.

Three instructions, exactly

9,585.7 → 9,588.7 retired per event, over three runs. The three flushes; nothing else in the loop moved.

Benchmarks with clflush



+CLFLUSHOPT AGAINST +PREFETCH

p50 414 → 409 ns -1.2%

p90 710 → 705 ns -0.7%

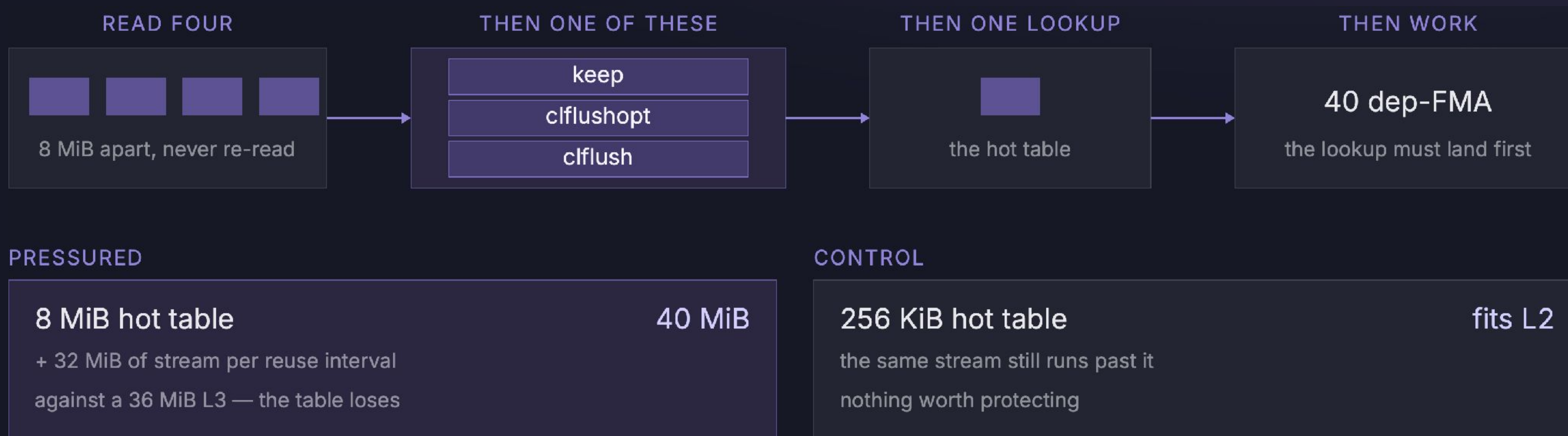
Instructions 9,814.7 → 9,820.7 +6 retired for three flushes

L1 hits 817.3 → 819.5 +2.2, the three lines they find

Stall cycles 634 → 495 waiting on an L1D miss, -21.9%

Three instructions in, two more L1 hits out, and a fifth of the time spent waiting on a miss goes with them.

Independent experiment to verify clflushopt



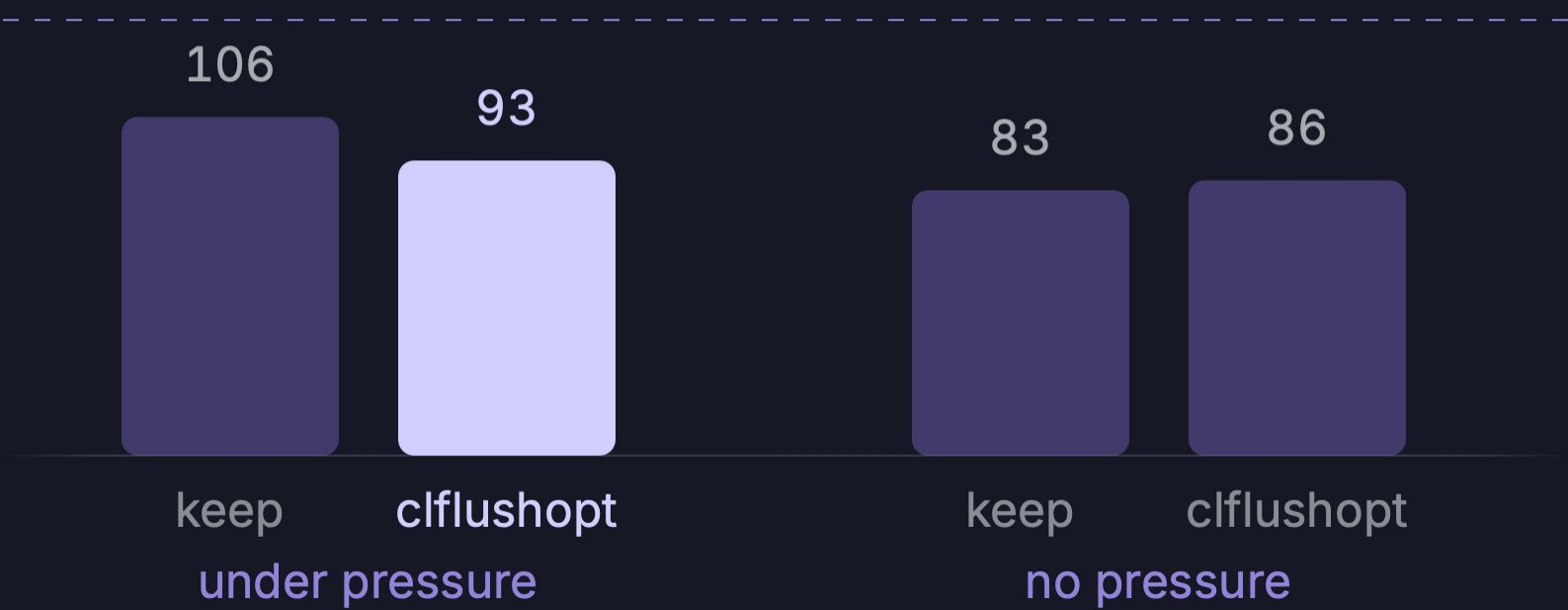
Four stream lines an event, read once and never again — then one lookup in a table every event needs.

The three arms differ by a single statement, and the hot table is sized either to feel the pressure or not to.

It only pays when the cache is full

ns per event

clflush · 137 ns in both



CLFLUSHOPT AGAINST KEEPING THE LINES

Under pressure 106.4 → 92.6 ns -12.9%

No pressure 83.2 → 86.4 ns +3.8%, a small cost

Stall cycles 473 → 432 pressured, -8.7%

L3 misses 4.13 → 3.69 per event, -11%

Fill buffers busy 0.8% → 4.0% they pipeline

With nothing worth protecting, the flush is pure overhead.

CLFLUSH, PRESSURED OR NOT

807 cycles either way · it serialises

Multi-core

We optimised MLP and cache eviction. Let's try making the application multi-core.

The walk is parallel

64 independent updates. Hand the tail of the holdings to a second core and both run the same loop over different options.

The event is not

Poll the ring, look the stock up, work out the move — one dependent chain nothing can prefetch. Main keeps that, and the larger slice with it.

The handoff is one line

Ids, deltas and the move travel together, so the aux core never has to read the stock record — the dependent hop the split exists to avoid.

THE SPLIT 24 of 64 holdings · to the aux core

Splitting one event across two cores



Main publishes the Req, Aux publishes the Res

Producer — the main core

```
for (size_t j = 0; j < ka; ++j)
    req.give[j] = st->holds[K - ka + j];
req.move = move;
req.n     = ka;
req.seq.store(seq, release); // hand off

update_span(st->holds, K - ka, move);

while (res.seq.load(acquire) != seq) {}
for (uint32_t i = 0; i < res.n; ++i)
    fold(res.hit[i]);
```

Consumer — the aux core

```
while (req.seq.load(acquire) != seq)
    {}

n = update_span(req.give, req.n, req.move);

for (uint32_t i = 0; i < n; ++i)
    res.hit[i] = hit[i];
res.n = n;
res.tsc = __rdtsc();
res.seq.store(seq, release); // hand back
```

Why the handoff is shaped this way



The ids and deltas travel together

Otherwise the aux would have to read the stock record itself — the dependent hop the handover exists to avoid.

Only one Req & Res

An event is finished before the next is looked at, so the aux always has one Req to compute on every event.

Main keeps the larger share

24 of 64 go to the aux. This ensures aux finishes before main thread finishes compute.

A second core improves median by 11%

nanoseconds per event

— p50 — p90



Handing **24 of the 64 holdings** to a second core takes the median 409 → **364 ns (−11.0%)** and the p90 705 → 575 (**−18.4%**) — both move together, so the whole distribution shifts. Main's own span falls 500 → 365 ns against the aux's 298, so main still finishes last.

Main hands over, then prices its own slice

```
// hand the aux its slice of the book
g_req.move = move;
g_req.seq.store(seq, release);

// then price main's own 40 holdings
update_span(holds, K - ka, g_req.move);
```

Anything wrong with that?

Let's compare 2 versions

Before — read back out of the buffer

```
// hand the aux its slice of the book
g_req.move = move;
g_req.seq.store(seq, release);

// then price main's own 40 holdings
update_span(holds, K - ka, g_req.move);
```

After — read it from main's own copy

```
// hand the aux its slice of the book
g_req.move = move;
g_req.seq.store(seq, release);

// then price main's own 40 holdings
update_span(holds, K - ka, move);
```

THE WHOLE DIFFERENCE

one load · one instruction

Taking the load out speeds up the event

nanoseconds per event

— p50 — p90



Dropping the read-back takes main's own span 365 → 359 ns and the event's median 364 → **344 ns (-5.5%)**. Main still finishes last, so the saving reaches the event.

The penalty of shared buffer read

Why that load is not free

Main wrote that line a few instructions earlier and the aux only ever reads it, so the load looks free.

But the aux polls the handover flag in the same line. A peer's read of a *Modified* line largely takes it away rather than leaving the owner a copy — so main's own value comes back from the peer, not from L1.

TAKING THAT ONE LOAD AWAY

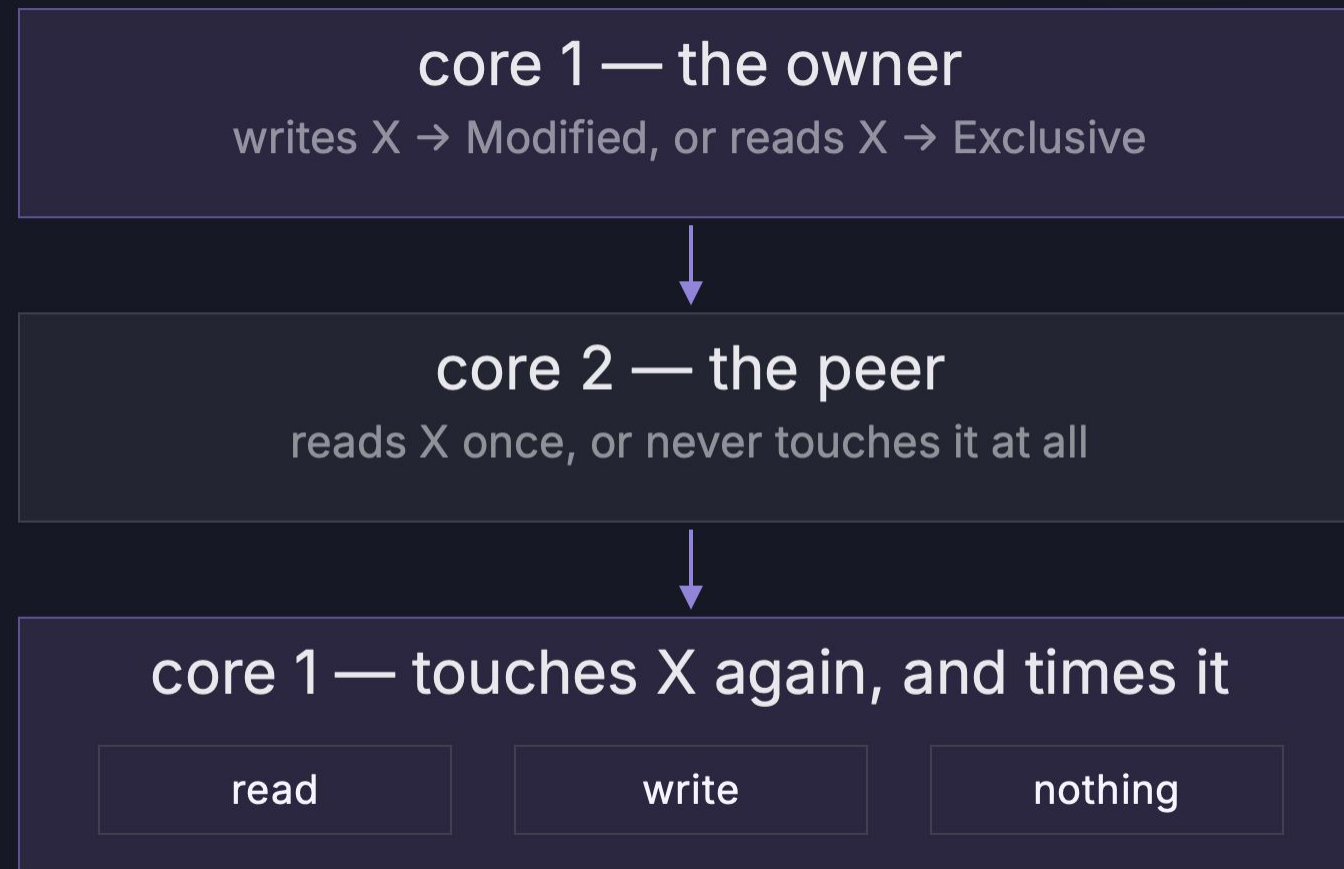
p50	364 → 344 ns -5.5%
p90	575 → 555 ns -3.5%
Main's own span	365 → 359 ns -1.6%
Snoop forwards	0.96 → 0.82 per event

How much does it cost to read one cache line?



Those four numbers were measured with one core running. L1 and L2 are private, so nothing there has changed — but L3 is common, and what a line costs now depends on who else has a copy of it.

One line, different MESI states → different R latency



One round of the experiment

```
// core 1 — leave X in a known state
if (dirty) *p += 1.0;      // Modified
else      sink += *p;     // Exclusive
go.store(r + 1, release); // peer may look

// core 2 — read it, or leave it alone
if (peer_reads) sink += *p;
done.store(r + 1, release);

// core 1 — one touch, and only that
ticks += timed_touch<OP2>(p) - overhead;
```

It depends on the peer + reader

ns for one access



ONE LINE, TWO CORES, 4,096 ROUNDS

Read, alone	0.81 ns 2.6 ticks
Peer read, clean	0.97 ns E → S, no change
Peer read, dirty	27.70 ns 34×
Write, peer copy	53.82 ns vs 2.80 ns alone
Empty window	0.59 ns the floor

NO CAPACITY MISSES

L3 misses 0.0 · all of this is coherence

When the owner reads it back

OWNER HAD	THE PEER	OWNER NOW	WHY	READ COSTS
M	never looks at it	M	the line never left L1	0.81 ns
M	reads the line	I	dirty data had to be handed over	27.70 ns
E	never looks at it	E	clean and unshared, still in L1	0.68 ns
E	reads the line	S	a clean line can simply be shared	0.97 ns

SHARING A CLEAN LINE

free · taking back a dirty one costs 34×

When the owner writes it instead

OWNER HAD	THE PEER	INVALIDATE?	WHY	WRITE COSTS
M	never looks at it	no	already the sole owner	2.80 ns
M	reads the line	yes	the peer's copy must go first	53.82 ns
E	never looks at it	no	E → M is a silent upgrade	2.60 ns
E	reads the line	yes	the peer's copy must go first	53.83 ns

READ-FOR-OWNERSHIP PER ACCESS 1.0 → 2.0 · and one cross-core snoop

Probing another core is expensive ?

Both the 27.7 ns read and the 53.8 ns write went off-core to ask a question. So, the latency would depend on

How far away it is

Whether we have to go in other cores
L1, L2 or we find the cache line in
shared L3 itself

Whether it is dirty

Whether the cacheline has been
updated and needs to be written

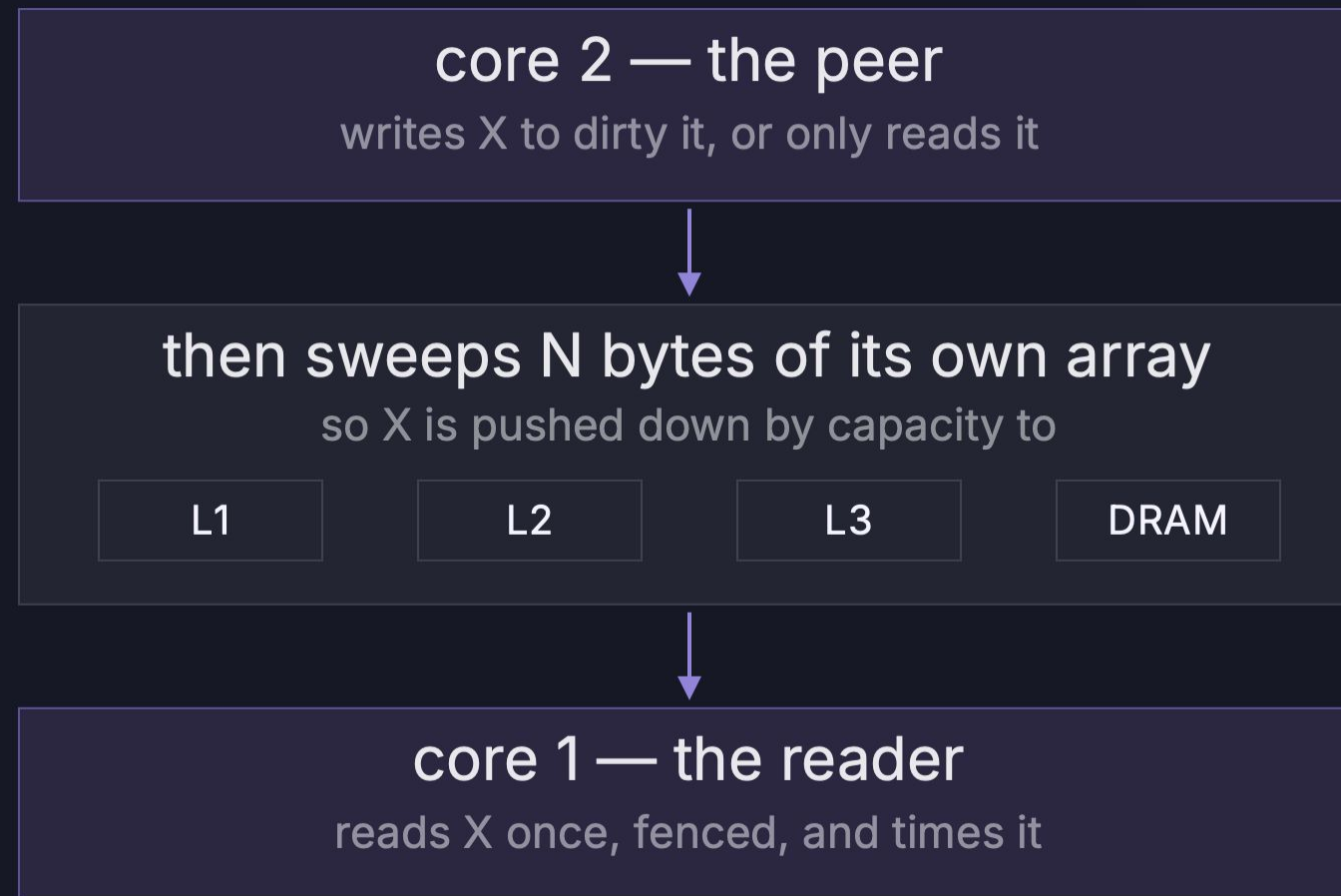
How many hold it

Whether its in an exclusive/modified
state and needs to be invalidated

WHAT IS TIMED

one load · the reader touches the line once

Place the line, then time one load



One round of the experiment

```
// core 2 — put the line somewhere
if (dirty) *p += 1.0;    // Modified
else      sink += *p;    // Exclusive

for (size_t k = 0; k < sweep; ++k)
    a += wsweep[sidx[base+k] & smask];
w_ready.store(r + 1, release);

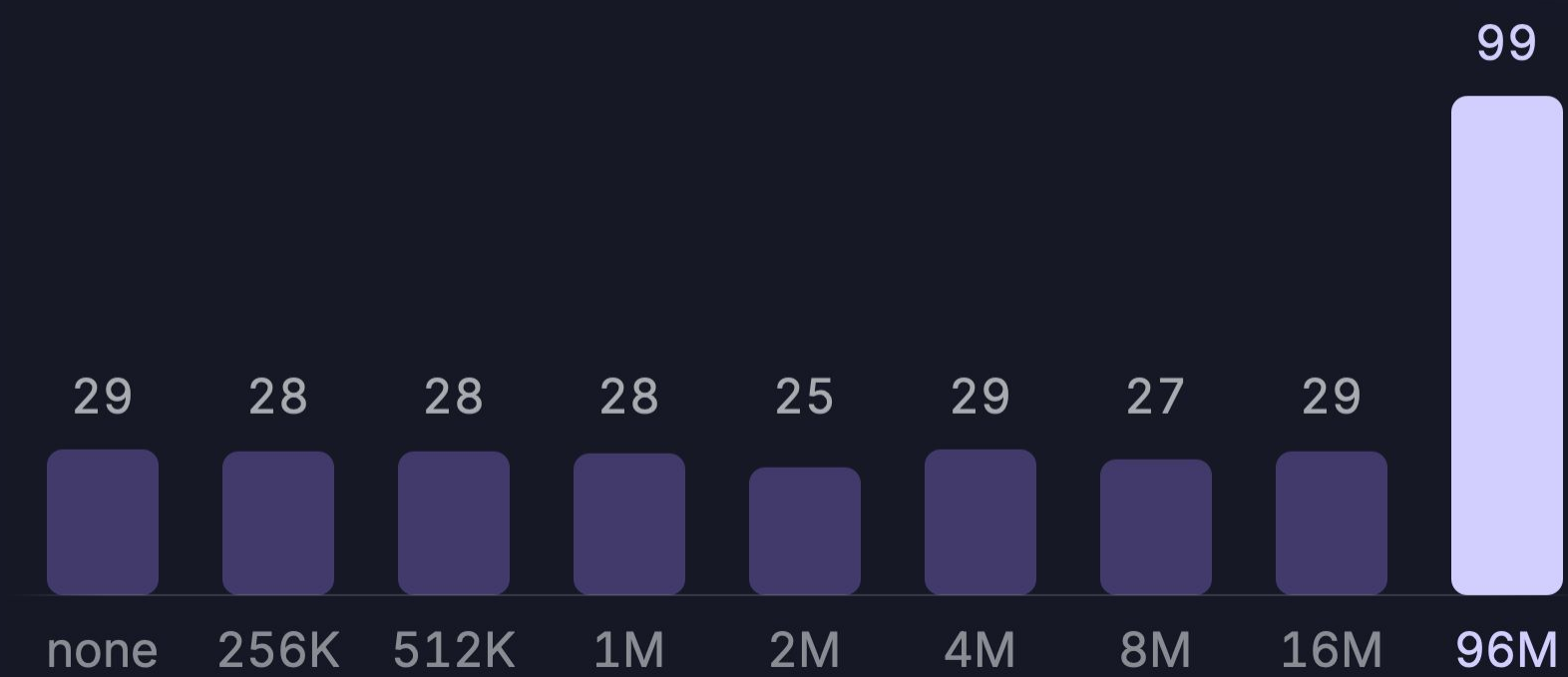
// core 1 — time exactly one load
while (w_ready.load(acquire) < r+1) {}
ticks += timed_load(p);    // one load
```

NOTHING IS ASSUMED ABOUT TIMING

a two-way handshake, every round

A clean line cost

ns for the one load · the peer's sweep



A CLEAN LINE, AND ONE THAT IS SHARED

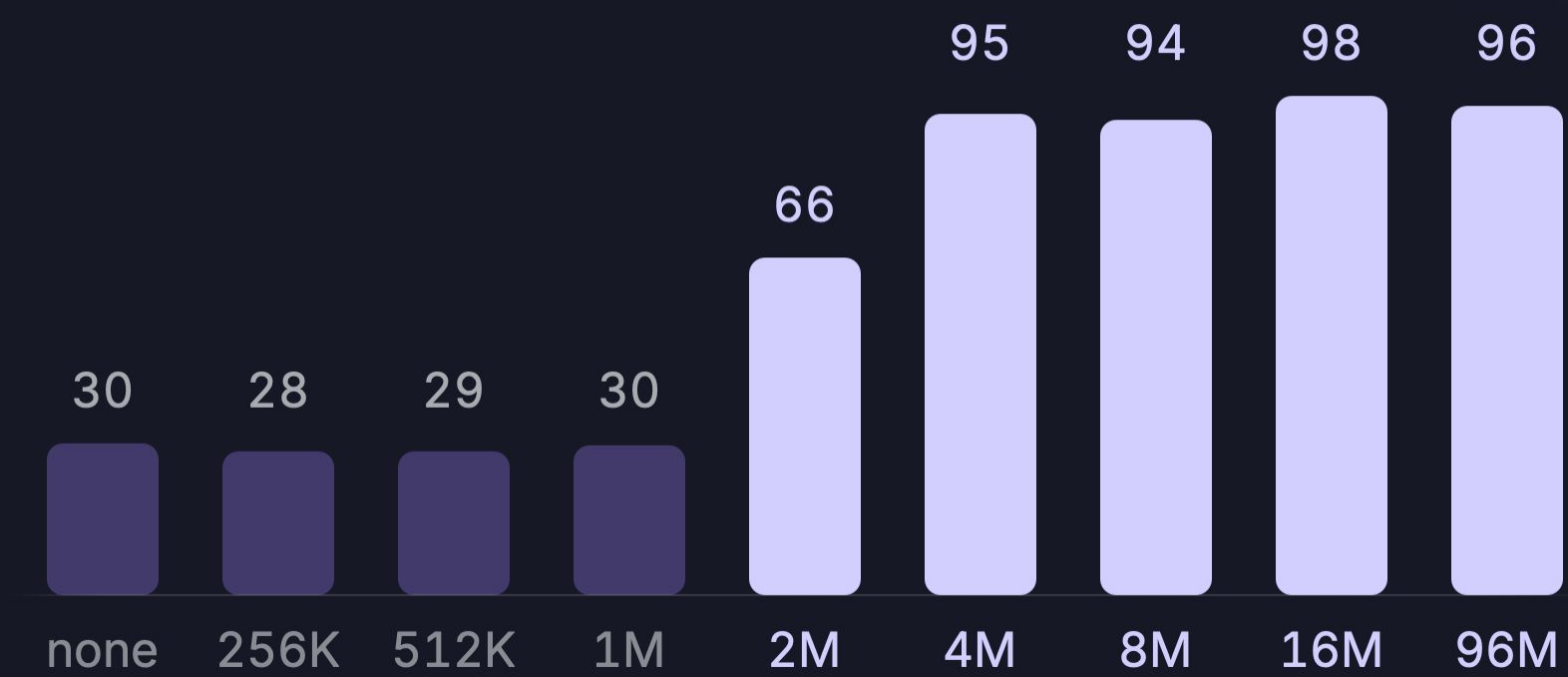
In the peer's L1	28.8 ns no sweep at all
Anywhere in L1-L3	25-29 ns eight sweeps
Peer's L3 gone too	98.8 ns now from DRAM
Two peers hold it	12.7 ns L3 answers directly
Apparatus floor	0.8 ns fences + an L1 hit

TWO SHARERS BEAT ONE OWNER

12.7 ns against 28.8 · no snoop at all

A dirty line cost

ns for the one load · the peer's sweep



A DIRTY LINE, THE SAME NINE SWEEPS

Still in L1 or L2	28–30 ns same as clean
Past the peer's L2	94.8 ns 3.3×
L3 misses	0.03 → 0.96 per load
Snoop forwards	0.96 → 0.00 nobody holds it
And the 96 MiB sweep	96.4 ns no worse than 4 MiB

THE CLEAN LINE STAYED IN L3

the dirty one went to DRAM instead

But where did the dirty line go?

At the 4 MiB sweep both lines sit in the same place, evicted from the peer's L2. One is answered by L3. The other is not.

CLEAN, READ

29.0

nanoseconds

L3 hit 0.77 — the line is in L3

DIRTY, READ

94.8

nanoseconds

L3 miss 0.96 — the line is not

THE GAP

3.3

times

same sweep, same level

SNOOP FORWARDS

0.00

per load

no core is holding it either

NEXT

why a dirty line evicted from L2 skips L3

The L3 is neither inclusive nor exclusive

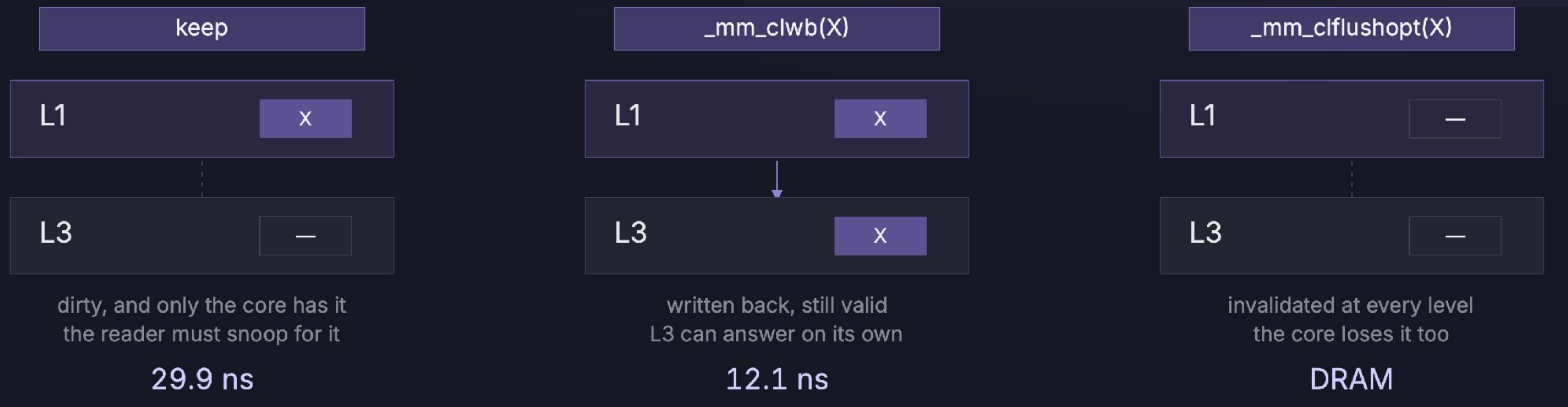


An inclusive L3 made every snoop cheap; an exclusive one trades that away for capacity. Intel's client L3 is *neither* : a line in L2 may or may not have a copy below it, and when L2 evicts one, whether L3 allocates for it is a heuristic — undocumented, and free to change. A clean victim was kept. The dirty one was written past L3 to memory.

A clean victim stays. A dirty one leaves.

How can we use that in the application?

CLWB



The two instructions have the same shape — weakly ordered, several in flight through the fill buffers, an *mfence* when something must observe them. They differ in what they leave behind: *clflushopt* drops the line, *clwb* writes it back and keeps it. That is why the flush suits the read-once ring, and the writeback suits a line the other core is about to read.

clwb only moves a dirty line

ns for the one load · no sweep



CLWB ON EACH, AGAINST KEEPING THE LINE

Dirty line	29.9 → 12.1 ns -60%
Clean line	28.4 → 26.2 ns -8%
Snoop, dirty	FWD → NONE L3 answers · 1.0 hit
Snoop, clean	FWD → NOFWD still asked · no L3 hit
Two peers sharing	12.5 ns where the dirty line landed
A clean writeback has nothing to write, so the line never leaves the peer.	

WHY THE CLEAN LINE BARELY MOVES

the writeback had nothing to write

So push the result list to L3

Before — raise the flag and stop

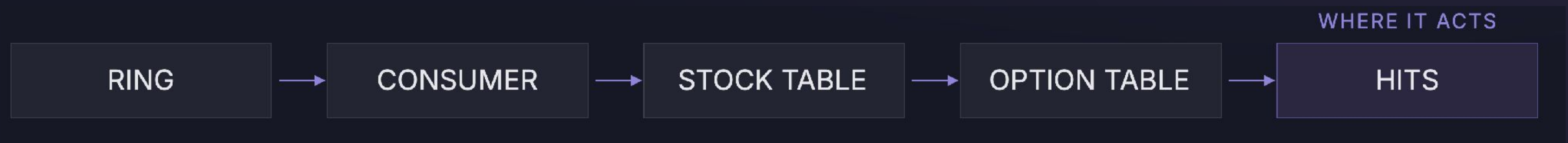
```
for (uint32_t i = 0; i < n; ++i)
    res.hit[i] = hit[i];
res.n = n;
res.tsc = __rdtsc();
res.seq.store(seq, release);
```

After — push it to L3 first

```
for (uint32_t i = 0; i < n; ++i)
    res.hit[i] = hit[i];
res.n = n;
res.tsc = __rdtsc();
res.seq.store(seq, release);
```

```
for (size_t i = 0; i < sizeof(res); i += 64)
    _mm_clwb((char*)&res + i);
_mm_mfence();
```

Push, so the reader need not snoop



Only aux → main is pushed

Similar optimisations can be explored
in main → aux path as well

Main stops snooping the aux

Snoop-forwards fall 0.87 → 0.34 per
event and plain L3 hits rise 2.53 →
3.07 — the line waits in L3 instead of
in the other core's L1.

It shortens the wait, not the
work

Main's wait on the aux drops 111 → 97
ns. The aux does the same arithmetic
either way.

The final conclusion: 510 ns down to 329

nanoseconds per event

— p50 — p90



Each improvement iteratively improves a simple application

Three things worth taking away

The compute is exactly same — only cache behaviour was modified.

Prefetch

Need to optimise the prefetching distance and the lines to prefetch based on application semantics and perf metrics

Clflushopt

clflushopt translates to just one instruction, frees up cache space and reduces stalls.

Coherency

Each shared read has a cost. So, benchmark and ensure minimal coherency traffic and *c/wb* it to L3 before any other core reads.

SAME ANSWERS, SAME MACHINE

510 ns → 329 ns per event

Questions?

Link to code repository :- <https://github.com/0xsanchit/CacheLatency>